



UNIVERSIDADE FEDERAL DE ALAGOAS
INSTITUTO DE COMPUTAÇÃO
BANCO DE DADOS



LUCAS GUTERRES BUENO
NÍCOLLAS PROVATTI GOMES
THÉO LUIZ RODRIGUES DA SILVA

Projeto Final

Maceió, maio de 2026

LUCAS GUTERRES BUENO
NÍCOLLAS PROVATTI GOMES
THÉO LUIZ RODRIGUES DA SILVA

Projeto Final

Relatório requerido como requisito
avaliativo da disciplina ECOM41 - Banco
de Dados Prof. Dr. Fabio Jose Coutinho da
Silva

Maceió, maio de 2026

Sumário

1. FUNCIONALIDADES E ARQUITETURA DO NEO4J.....	4
2. MODELO DE DADOS.....	4
3. LINGUAGEM DE MANIPULAÇÃO E DEFINIÇÃO: CYPHER.....	5
3.1 Operações de DML (Data Manipulation Language) - O ciclo CRUD.....	6
3.2 Operações de DDL (Data Definition Language).....	7
4. PROCESSAMENTO DE TRANSAÇÕES: GARANTIAS ACID.....	7
5. SEGURANÇA: CONTROLE DE ACESSO E PROTEÇÃO DE DADOS.....	8
5.1 Controle de Acesso Baseado em Funções (RBAC).....	8
5.2 Autenticação e Federação de Identidade.....	8
5.3 Criptografia de Dados.....	9
5.4 O Grafo como Motor de Autorização.....	9
6. OTIMIZAÇÃO: ÍNDICES NATIVOS, RESTRIÇÕES E PLANOS DE EXECUÇÃO.....	9
6.1 Índices Nativos para Pontos de Partida (Âncoras).....	10
6.2 Restrições (Constraints) de Integridade Estrutural.....	10
6.3 Análise de Desempenho e Planos de Execução (EXPLAIN e PROFILE).....	10
7. YELP OPEN: DATASET ESCOLHIDO.....	11
7.1 Arquivos e Variáveis Utilizadas.....	11
7.2 Processo de Limpeza e Filtragem.....	11
7.3 Composição Final e Balanceamento.....	12

1. FUNCIONALIDADES E ARQUITETURA DO NEO4J

O Neo4j destaca-se pelo seu desempenho otimizado e arquitetura projetada especificamente para o tratamento de dados altamente conectados. Diferentemente dos bancos de dados relacionais, onde a complexidade exige múltiplos JOINS, o processamento de grafos no Neo4j é nativo e fundamentado no princípio da "adjacência livre de índices" (index-free adjacency). Nesta arquitetura, cada nó mantém referências físicas diretas para os seus nós adjacentes. Enquanto junções através de índices globais em bancos tradicionais geralmente custam $O(\log n)$ por operação, atravessar um relacionamento no Neo4j é uma operação de tempo constante, ou $O(1)$. O tempo de execução é proporcional apenas ao tamanho da sub-rede percorrida, garantindo alta performance.

Apoiado nessa arquitetura de alta performance, o Neo4j oferece um ecossistema robusto de funcionalidades voltadas para a manipulação e análise de dados complexos:

- **Graph Data Science (GDS) Nativo:** O banco possui uma biblioteca integrada para execução de algoritmos complexos em larga escala diretamente nos dados armazenados. Isso permite rodar algoritmos de caminho mínimo (como Dijkstra ou A* para roteamento em malhas e mapas), centralidade e detecção de comunidades (extremamente úteis para caracterização de dados de tráfego urbano e fluxos em redes).
- **Visualização Interativa Integrada:** Através de ferramentas como o *Neo4j Browser* e o *Neo4j Bloom*, a plataforma permite a exploração visual e interativa dos nós e relacionamentos. Os dados não retornam apenas em formato tabular, mas são renderizados graficamente, facilitando o reconhecimento de padrões.
- **Suporte Amplo a Integrações e Drivers:** O banco oferece drivers oficiais otimizados para diversas linguagens, com destaque para um suporte muito maduro em Python. Isso permite integrar o banco de grafos diretamente em pipelines de processamento de dados e scripts de análise, facilitando a extração para bibliotecas como Pandas e Matplotlib.
- **Importação Massiva de Dados (ETL):** Funcionalidades como o **LOAD CSV** e o *Neo4j Data Importer* permitem que grandes volumes de dados tabulares (datasets estruturados) sejam rapidamente mapeados e convertidos para o modelo de grafos sem a necessidade de ferramentas externas complexas.

2. MODELO DE DADOS

O Neo4j fundamenta o seu armazenamento e a sua representação de dados no modelo de Grafo de Propriedades Rotulado (*Labeled Property Graph*). Do ponto de vista acadêmico e arquitetônico, esse modelo destaca-se por reduzir significativamente a

incompatibilidade de impedância (*impedance mismatch*) entre a análise do problema e a sua implementação.

Este modelo de dados é composto por blocos estruturais fundamentais:

- **Nós (Vértices):** Representam as entidades principais do domínio (ex: uma Pessoa, um Local, um Produto).
- **Rótulos (Labels):** São as categorias ou classes atribuídas aos nós. Eles servem para agrupar nós semelhantes e criar índices. Um mesmo nó pode ter múltiplos rótulos (ex: um nó pode ser rotulado como **Pessoa** e **Estudante** simultaneamente).
- **Relacionamentos (Arestas):** São cidadãos de primeira classe no banco de dados. Eles conectam os nós, estruturando o grafo. Todo relacionamento possui, obrigatoriamente, uma direção, um tipo (nome único) e conecta um nó de origem a um nó de destino. A direção e o tipo adicionam semântica clara à rede (ex: **(:Pessoa)-[:ESTUDA_N0]->(:Instituto)**).
- **Propriedades:** Tanto os nós quanto os relacionamentos podem conter propriedades armazenadas na forma de atributos do tipo chave-valor (ex: **nome: 'João', ano_ingresso: 2022**). As propriedades nos relacionamentos são cruciais para expressar metadados, como a força, o tempo, a qualidade ou o custo (peso) daquela conexão.

A lógica dessa modelagem por nós e arestas é muito forte para a resolução de problemas de redes, mapas e rotas espaciais ou logísticas. Podemos usar a analogia da navegação espacial de um robô autônomo dentro de um armazém: neste cenário, os cruzamentos (pontos de decisão) são modelados como nós, e os corredores que ligam esses cruzamentos são os relacionamentos. O comprimento ou o tempo de travessia do corredor é salvo como uma propriedade do relacionamento.

Como os dados já estão dispostos fisicamente nessa estrutura de rede, o modelo torna-se perfeito para rodar algoritmos clássicos de grafos diretamente dentro do banco de dados. A execução de buscas para descobrir o caminho mínimo (*shortest path*) — utilizando algoritmos nativos como o Algoritmo de Dijkstra ou o Algoritmo A* — opera de maneira altamente otimizada, calculando as rotas ao navegar diretamente pelas adjacências físicas dos nós.

3. LINGUAGEM DE MANIPULAÇÃO E DEFINIÇÃO: CYPHER

Diferentemente dos bancos de dados relacionais que operam baseados na linguagem SQL (*Structured Query Language*), o Neo4j utiliza o Cypher. O Cypher é uma linguagem de consulta declarativa, expressiva e compacta, projetada especificamente para o modelo de grafos de propriedades. A grande vantagem acadêmica e prática dessa

linguagem é a redução drástica da diferença entre a concepção teórica do problema e a implementação do código.

Uma das características arquitetônicas mais marcantes do Cypher é a sua sintaxe visual, que emprega "ASCII-Art" para desenhar e reconhecer padrões topológicos na rede. Essa abordagem segue o princípio da "especificação por exemplo": em vez de descrever proceduralmente como o banco deve navegar, o usuário simplesmente desenha a estrutura do grafo que deseja buscar.

Na sintaxe do Cypher, as topologias são mapeadas da seguinte forma:

- **Nós (entidades):** São representados entre parênteses `()`, simulando visualmente os círculos ou caixas de um diagrama.
- **Relacionamentos (arestas):** São desenhados usando traços e setas `-->` ou `<--` para denotar a direção. O tipo do relacionamento é declarado dentro de colchetes, precedido por dois pontos, como em `[ :RELACIONAMENTO ]`.

Sendo assim, o padrão de comunicação nativo no banco obedece à estrutura visual `(nó)-[:RELACIONAMENTO]->(outro_nó)`. Para materializar isso, uma busca por duas pessoas que se conhecem pode ser declarada de forma altamente legível: `(a:Person)-[:KNOWS]->(b:Person)`.

3.1 Operações de DML (Data Manipulation Language) - O ciclo CRUD

Por ser uma linguagem robusta de manipulação, o Cypher implementa o ciclo CRUD completo através de cláusulas declarativas:

- **Create (C):** Executado através da cláusula `CREATE` para instanciar novos nós e relacionamentos, ou através do `MERGE` (que funciona como um *upsert*, criando a estrutura apenas se ela não existir no banco, evitando duplicidade).
- **Read (R):** Controlado pela cláusula `MATCH`, que atua como o motor central para busca de padrões topológicos, frequentemente combinado com o `WHERE` para filtragem de atributos e finalizado com o `RETURN`.
- **Update (U):** Realizado através da cláusula `SET`, utilizada para adicionar ou modificar propriedades de nós e relacionamentos já existentes na rede.
- **Delete (D):** Feito através da cláusula `DELETE`. No Neo4j, um nó não pode ser deletado se ainda possuir conexões físicas; para contornar isso com segurança, utiliza-se a cláusula `DETACH DELETE`, que remove os relacionamentos do nó e o nó em si em uma única operação atômica.

3.2 Operações de DDL (Data Definition Language)

Embora o Neo4j seja um banco *schema-optional* (onde a estrutura se molda conforme os dados são inseridos), o Cypher possui comandos específicos de DDL para garantir a integridade e a performance do sistema através da criação de índices e restrições:

- **Constraints (Restrições):** Permitem impor regras de integridade nos dados, como garantir a unicidade de uma propriedade em um nó (ex: `CREATE CONSTRAINT FOR (p:Pessoa) REQUIRE p.id IS UNIQUE`).
- **Indexes (Índices):** Utilizados para acelerar a busca inicial de nós no grafo antes de iniciar a travessia física de relacionamentos (ex: `CREATE INDEX FOR (p:Pessoa) ON (p.nome)`).

4. PROCESSAMENTO DE TRANSAÇÕES: GARANTIAS ACID

No ecossistema de bancos de dados modernos, uma característica marcante de muitas soluções NoSQL é a flexibilização da consistência imediata em prol da escalabilidade e resiliência. Nesses sistemas, as propriedades ACID tradicionais costumam ser substituídas pelo modelo BASE (*Basic Availability, Soft-state, Eventual consistency*). O Neo4j, entretanto, diferencia-se fundamentalmente nesse aspecto.

Construído como um sistema de processamento transacional online (OLTP), o Neo4j não abre mão da consistência e foi projetado para garantir conformidade total com as propriedades **ACID (Atomicidade, Consistência, Isolamento e Durabilidade)**, oferecendo um ambiente seguro para a manipulação de redes complexas.

Do ponto de vista técnico e arquitetônico, esse processamento ocorre da seguinte forma:

- **Atomicidade:** Todas as operações contidas em uma transação Cypher são tratadas como uma única unidade de trabalho. Se uma inserção falhar no meio de um script complexo de criação de nós, toda a operação sofre *rollback* e o grafo retorna ao seu estado original estável.
- **Consistência:** O Neo4j garante que o grafo passe de um estado válido para outro estado válido, respeitando todas as restrições (*constraints*) e regras de integridade estrutural (como a obrigatoriedade de direções em relacionamentos).
- **Isolamento:** O Neo4j opera nativamente sob o nível de isolamento *Read Committed*. Isso significa que as alterações feitas dentro de uma transação permanecem invisíveis para outras sessões até que o *commit* seja realizado. O gerenciamento de concorrência aplica bloqueios de escrita (*write locks*) nos nós e relacionamentos afetados, tratando cenários de *deadlock* e serializando escritas.

concorrentes, enquanto implementa leituras não-bloqueantes (*non-blocking reads*) para manter alta performance de consulta.

- **Durabilidade:** Assim que uma transação é confirmada, suas alterações são registradas imediatamente em disco por meio de um mecanismo de log de transações conhecido como *Write-Ahead Log* (WAL). Mesmo em caso de falha catastrófica de energia ou hardware logo após o *commit*, o Neo4j é capaz de recuperar e reconstruir o estado exato dos dados no momento da reinicialização.

5. SEGURANÇA: CONTROLE DE ACESSO E PROTEÇÃO DE DADOS

No ecossistema corporativo, o Neo4j Enterprise fornece uma arquitetura de segurança robusta baseada em três pilares principais de governança e proteção de dados, além de servir como uma excelente ferramenta lógica de autorização.

5.1 Controle de Acesso Baseado em Funções (RBAC)

O gerenciamento de acessos ocorre por meio de um modelo RBAC nativo, onde os privilégios são concedidos a funções (*Roles*) atribuídas aos usuários. O sistema conta com quatro funções nativas padrão:

- **admin:** Controle total sobre o SGBD, usuários, funções e configurações.
- **architect:** Permissão de leitura, escrita de dados e modificação de esquemas/DDL (como criação de índices e *constraints*).
- **publisher:** Permissão de leitura e escrita de dados, mas sem autorização para alterar o esquema.
- **reader:** Permissão estrita de apenas leitura (*Read-Only*) sobre o grafo.

Granularidade ao nível de subgrafo (*Sub-graph Granularity*): Permite definir privilégios extremamente específicos para os usuários:

- **TRAVERSE:** Permite navegar por um nó ou aresta para alcançar outra parte do grafo, mesmo sem permissão de ler as propriedades do elemento.
- **READ:** Permite a leitura de propriedades específicas baseada nos rótulos (*labels*).
- **WRITE:** Permite a mutação de elementos estritamente delimitados.

5.2 Autenticação e Federação de Identidade

Para integração com arquiteturas de TI empresariais, o Neo4j suporta a centralização de credenciais por meio de plug-ins externos, com suporte nativo a protocolos como **LDAP** e **Microsoft Active Directory (AD)**.

- **Mapeamento de Grupos:** Permite delegar a verificação de senhas ao servidor externo e mapear grupos do AD diretamente para as *Roles* internas do Neo4j (ex: usuários do grupo "TI_DBAS" no AD ganham a função `admin` automaticamente).
- **Single Sign-On (SSO):** O suporte a Kerberos e SSO garante um acesso fluido e seguro dentro da rede corporativa.

5.3 Criptografia de Dados

O Neo4j assegura a conformidade com regulamentações rigorosas de privacidade através de dois mecanismos:

- **Criptografia em Trânsito (*Data-in-Transit*):** Protege toda a comunicação da rede (entre aplicações cliente como drivers Python, instâncias do cluster ou ferramentas de administração) nativamente via TLS através de HTTPS e do protocolo Bolt+SSC.
- **Criptografia em Repouso (*Data-at-Rest*):** Garante a encriptação dos arquivos físicos do banco e logs de transação gravados no disco rígido usando o algoritmo AES-256, integrado a sistemas de gerenciamento de chaves externos (KMS).

5.4 O Grafo como Motor de Autorização

Além dos recursos de infraestrutura, o próprio modelo orientado a grafos se destaca como uma base arquitetônica ideal para construir sistemas complexos de autorização de software. Enquanto bancos relacionais sofrem com a lentidão de múltiplas tabelas e heranças de permissões (o problema conhecido como *join pain*), o Neo4j executa essas varreduras em milissegundos. Sob a ótica da auditoria, isso permite:

- **Análise Top-down:** Descobrir rapidamente todos os recursos, estruturas ou serviços que um usuário e seus respectivos grupos podem visualizar ou gerenciar.
- **Análise Bottom-up:** A partir de um recurso crítico e sensível no sistema, rastrear perfeitamente de volta à origem quem são todos os usuários com permissão para modificá-lo.

6. OTIMIZAÇÃO: ÍNDICES NATIVOS, RESTRIÇÕES E PLANOS DE EXECUÇÃO

Para garantir alta performance em cenários de grande volume de dados e consultas complexas, o Neo4j combina a sua arquitetura nativa de "adjacência livre de índices" com ferramentas avançadas de otimização de consultas e governança de esquema.

6.1 Índices Nativos para Pontos de Partida (Âncoras)

Embora a travessia entre nós já conectados seja uma operação de custo constante $O(1)$, o passo inicial de qualquer consulta no Cypher exige localizar um "nó de partida" ou âncora na rede. Sem um índice, o banco precisaria realizar uma varredura completa (*Full Graph Scan*) por todos os nós, gerando um custo linear $O(n)$.

Para reduzir esse custo inicial para $O(\log n)$, o Neo4j adota estratégias de indexação nativa:

- **Índices de Intervalo (*Range Indexes*):** Substitutos dos antigos índices *B-Tree*, são os índices padrão utilizados para buscas de igualdade, intervalos numéricos e ordenação de propriedades.
- **Índices de Texto Completo (*Fulltext Indexes*):** Otimizados para buscas textuais complexas, correspondências parciais e termos semelhantes em strings longas.
- **Índices de Busca (*Lookup Indexes*):** Índices internos automáticos que otimizam a localização de nós e relacionamentos puramente com base em seus rótulos (*labels*) ou tipos, independentemente das propriedades.

Ao utilizar essas estruturas, o banco localiza o ponto inicial em milissegundos e, a partir dele, realiza a navegação ultra rápida pelos ponteiros físicos das adjacências.

6.2 Restrições (Constraints) de Integridade Estrutural

Em conformidade com o modelo transacional ACID, o Neo4j permite embutir regras de integridade diretamente no motor do banco, evitando dados duplicados ou corrompidos sem depender exclusivamente da camada da aplicação:

- **Restrição de Unicidade (*Uniqueness Constraints*):** Garante que uma propriedade identificadora seja exclusiva para um determinado rótulo (ex: `CREATE CONSTRAINT FOR (p:Pessoa) REQUIRE p.id IS UNIQUE`).
- **Restrição de Existência (*Existential Constraints*):** Força que propriedades críticas sejam obrigatoriamente preenchidas no momento da criação do nó ou relacionamento (`IS NOT NULL`).
- **Chave de Nó (*Node Key Constraints*):** Combina as duas regras anteriores em uma única restrição. Garante que um nó possua obrigatoriamente um conjunto de propriedades e que essa combinação seja única no grafo.

6.3 Análise de Desempenho e Planos de Execução (EXPLAIN e PROFILE)

Escrever consultas declarativas em Cypher exige visibilidade sobre como o otimizador do banco interpretará o código. Para isso, o Neo4j disponibiliza duas ferramentas analíticas fundamentais que devem anteceder a query:

- **EXPLAIN:** Analisa a consulta e renderiza o plano de execução planejado pelo otimizador de consultas, mostrando os operadores lógicos que serão usados, mas sem executar a query fisicamente. É ideal para testar a estrutura de consultas pesadas em produção.
- **PROFILE:** Executa a consulta de fato e retorna o plano de execução real acompanhado de métricas físicas de desempenho. A métrica mais crucial analisada aqui são os *DB Hits* (acessos físicos ou lógicos do motor à memória ou ao disco).

A análise do **PROFILE** permite que engenheiros e cientistas de dados identifiquem gargalos, verifiquem se as consultas estão de fato utilizando os índices criados e eliminem travessias redundantes no grafo, refinando o código para máxima eficiência.

7. Yelp Open - Dataset Escolhido

Este conjunto de dados é uma coleção de informações sobre empresas, avaliações e usuários, originalmente disponibilizada em formato JSON. Abaixo estão os detalhes principais sobre a composição e o processamento deste dataset:

7.1 Arquivos e Variáveis Utilizadas

Embora o dataset original contenha seis arquivos principais (**business**, **review**, **user**, **checkin**, **tip** e **photo**), a pesquisa focou especificamente em dois deles:

- **business.json:** Contém dados das empresas, como localização, atributos e categorias.
- **review.json:** Contém o texto completo das avaliações, IDs de usuários e empresas, e a **classificação por estrelas (de 1 a 5)**.

7.2 Processo de Limpeza e Filtragem

Para tornar o dataset adequado ao objetivo do estudo (análise de sentimentos em restaurantes), os pesquisadores realizaram os seguintes filtros:

1. **Status do Negócio:** Foram selecionadas apenas empresas que ainda estavam abertas.
2. **Categoria:** O estudo restringiu-se exclusivamente à categoria de **restaurantes**.
3. **Localização:** Foram considerados apenas estabelecimentos localizados nos **Estados Unidos**.
4. **Idioma:** Utilizou-se uma biblioteca de detecção de linguagem para remover avaliações que não estivessem escritas em **inglês**.

7.3 Composição Final e Balanceamento

O pesquisador observou que o dataset original era **desbalanceado**, com uma predominância muito maior de avaliações positivas (4 e 5 estrelas) do que negativas. Para evitar vieses no modelo BERT, foi criado um dataset balanceado com o mesmo número de registros para cada categoria de estrelas.

Foram realizados experimentos com duas variações de tamanho:

- **Subconjunto de Pensilvânia (PA):** Devido a limitações de hardware no início da pesquisa, os dados foram restringidos a este estado, resultando em um dataset final de **92.201 avaliações** (aproximadamente 17.500 por categoria).
- **Dataset dos Estados Unidos:** Com acesso a um hardware mais potente, foi utilizado um conjunto maior abrangendo todo o país, totalizando **377.846 avaliações** (cerca de 70.000 por categoria).

Essa estrutura permitiu que o modelo BERT fosse treinado para identificar não apenas se uma avaliação era positiva ou negativa, mas também o **grau dessa polaridade** seguindo a escala de 1 a 5 estrelas do Yelp.